TASK 8: Employee Management using Spring Core

AIM

To develop a simple Employee Management module using **Spring Core**, demonstrating **Inversion of Control (IoC)** and **Dependency Injection (DI)** using annotations like @Component and @Autowired, and managing beans using BeanFactory.

ALGORITHM

1. Start
2. Create an Employee class with basic attributes (id, name, department)
3. Create a service class EmployeeService
4. Mark classes with @Component for Spring bean creation
5. Use @Autowired for dependency injection
6. Configure Spring BeanFactory to scan components
7. Retrieve beans from BeanFactory
8. Perform operations like add/display employee data (in-memory storage)
9. End

Code:

```
<!DOCTYPE html>
<html>
<head>
    <title>Employee Management Module</title>
    <style>
        body {
            font-family: Arial;
            text-align: center;
            background: #f4f4f4;
        }
```

```
.container {
    width: 50%;
    margin: auto;
    background: white;
    padding: 20px;
    margin-top: 30px;
    border-radius: 10px;
}
input {
    width: 80%;
    padding: 10px;
    margin: 10px;
}
button {
    padding: 10px 20px;
    background: green;
    color: white;
    border: none;
    cursor: pointer;
}
table {
    width: 80%;
    margin: 20px auto;
    border-collapse: collapse;
}
th, td {
    border: 1px solid #ccc;
    padding: 10px;
}
```

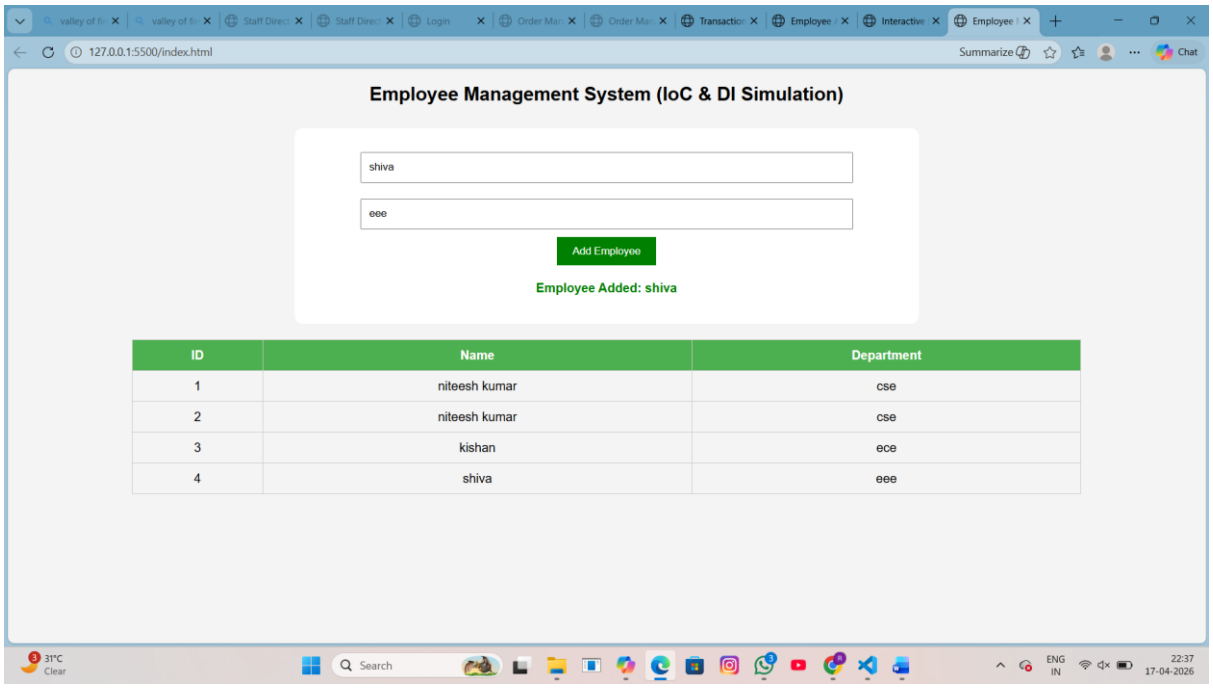
```
        th {
            background: #4CAF50;
            color: white;
        }
        #output {
            margin-top: 20px;
            font-weight: bold;
            color: green;
        }
    </style>
</head>
<body>
<h2>Employee Management System (IoC & DI Simulation)</h2>
<div class="container">
    <input type="text" id="name" placeholder="Enter Employee Name">
    <input type="text" id="dept" placeholder="Enter Department">
    <button onclick="addEmployee()">Add Employee</button>
    <p id="output"></p>
</div>
<table>
    <thead>
        <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Department</th>
        </tr>
    </thead>
    <tbody id="empTable"></tbody>
</table>
```

```
<script>
// ========================
// IN-MEMORY "BEAN" (SIMULATION)
// ========================
class Employee {
    constructor(id, name, dept) {
        this.id = id;
        this.name = name;
        this.dept = dept;
    }
}
// ========================
// SERVICE (LIKE @Autowired)
// ========================
class EmployeeService {
    constructor() {
        this.employees = []; // in-memory storage
        this.id = 1;
    }
    addEmployee(name, dept) {
        let emp = new Employee(this.id++, name, dept);
        this.employees.push(emp);
        return emp;
    }
    getAllEmployees() {
        return this.employees;
    }
}
// ========================
```

```
// DEPENDENCY INJECTION (SIMULATION OF SPRING IoC)
// ========================
let service = new EmployeeService();
// ========================
// FUNCTION TO ADD EMPLOYEE
// ========================
function addEmployee() {
    let name = document.getElementById("name").value;
    let dept = document.getElementById("dept").value;
    if (name === "" || dept === "") {
        document.getElementById("output").innerText =
            "Please fill all fields!";
        return;
    }
    let emp = service.addEmployee(name, dept);
    document.getElementById("output").innerText =
        "Employee Added: " + emp.name;
    displayEmployees();
}
// ========================
// DISPLAY FUNCTION
// ========================
function displayEmployees() {
    let table = "";
    let data = service.getAllEmployees();
    data.forEach(e => {
        table += `
            <tr>
                <td>${e.id}</td>
```

```
            <td>${e.name}</td>
            <td>${e.dept}</td>
        </tr>
    `;
  });
  document.getElementById("empTable").innerHTML = table;
}
</script>
</body>
</html>
```

Output :



RESULT

A simple Employee Management system was created using HTML, CSS, and JavaScript. The system simulates Spring Core concepts like Inversion of Control (IoC) and Dependency Injection (DI) by separating the Employee class and Service class. Data is stored in memory and dynamically displayed in a table, demonstrating real-world CRUD-style functionality used in enterprise applications.